

Price Sense AI - Write-Up

Prototype: <https://price-sense-ai-xlv53egdr3mquoteo5rslz.streamlit.app/>

Code: <https://github.com/vaishnavichalla-code/Price-Sense-AI>

Price Sense AI helps mid-market retailers answer one question before they discount: **"Should I run this promotion?"** The user describes a proposed promotion (product, discount, timing) and gets a clear recommendation - Run it / Consider it / Don't run it - backed by projected lift, cannibalization, incremental profit, and risk, plus a chart showing profit at every discount level so they can find the optimal discount.

1. How I decided the features in the POC

I started from the core pain in the brief: retailers run promotions on gut feel and can't tell whether a promotion actually made them money. So I anchored every feature to one job - turning a promo idea into a confident, profit-based decision - and cut anything that did not serve that job in 24 hours.

That gave me four must-have features:

1. **Simple promotion inputs** - product, discount %, and timing. This is the minimum a retailer needs to describe a promotion, and it keeps the tool approachable.
2. **A single, clear recommendation** - Run it / Consider it / Don't run it, in plain language with a one-line reason. A retailer should not need a data team to read it.
3. **The four numbers that justify the decision** - projected lift, cannibalization, incremental profit, and risk. These are exactly the questions the brief said retailers cannot currently answer.
4. **A "best discount" chart** - profit plotted across every discount level. This directly answers the brief's question "was 25% off better than 20%?" and turns the tool from a yes/no checker into an optimizer.

I deliberately left out account logins, multi-product baskets, and real data upload. They are valuable (see section 5) but not needed to prove the concept or demo the experience.

2. How I created the product

I built it by partnering with an AI coding assistant, which is the workflow the brief was evaluating. My role was to make the product and economic decisions; the assistant handled the implementation, and I tested and validated each step.

I chose **Streamlit** (a Python framework that renders a web UI from plain Python) because it let me produce a clean, demo-ready web app quickly without separate front-end work.

I built in small, testable increments: inputs first, then the recommendation engine, then the results and chart, then polish. After each step I ran the app and tried edge cases (very shallow and very deep discounts, high- and low-elasticity products) to confirm the behavior was believable before moving on.

The "intelligence" is **simulated but grounded in real promotion economics**, which the brief explicitly allows. Each product carries a small profile - price, cost, baseline weekly units, and price elasticity - and the engine computes:

- **Lift** from elasticity and discount depth (bigger, more price-sensitive discounts sell more units), amplified by timing.
- **Cannibalization** - a share of the extra volume is treated as stolen from other products or pulled forward from future weeks, so it is not counted as true growth.
- **Incremental profit** - profit in the promo week minus profit in a normal week minus the profit lost to cannibalization. This captures the key trap: you give up margin on every unit, including the ones you would have sold anyway at full price.
- **Risk** - rises with discount depth, thin margins, and cannibalization.

The recommendation is driven by incremental profit: clearly positive means Run it, marginal means Consider it, negative means Don't run it.

3. Technical architecture, and how it changes with scale

Today (prototype): a single Streamlit application with three clear layers:

- **Input layer** - the controls that capture the proposed promotion.
- **Recommendation engine** - a single, isolated **analyze()** function containing all the economic logic. Nothing about the UI leaks into it.
- **Presentation layer** - the recommendation banner, metric cards, and profit chart.

Product data lives in a **CATALOGS** structure that separates *data* (each industry's products and prices) from *logic* (the engine). This is what makes the product **horizontal**: supporting a new industry - a pharmacy, an electronics chain - is a single data entry with no code change.

At scale, the architecture stays the same and the components get upgraded:

- **Real intelligence**: the mocked elasticity and baseline numbers are replaced by a machine-learning model trained on the retailer's real point-of-sale history. Because the engine is already isolated in one function, this swap does not touch the UI.
- **Real data store**: the in-code catalog moves to a database holding thousands of SKUs with live prices and costs.
- **Service split**: the single app splits into a backend service (the engine, exposed as an API) and a frontend, so it can serve many concurrent customers and scale horizontally behind a load balancer, with caching for repeated queries.

The guiding principle throughout is that I separated logic, data, and UI - so scaling is mostly swapping components, not a rewrite.

4. Challenges I faced

The main challenge was making the simulated intelligence feel realistic. My first version recommended "Don't run it" for **every** promotion, because the profit model was too pessimistic - it never found a case worth doing. I tuned the assumptions (price elasticity and the cannibalization rate) until the tool produced a believable spread: shallow discounts on responsive products are profitable, deep discounts lose money, and a low-elasticity staple like milk should not be promoted at all. This reinforced the value of testing the product like a user before trusting it.

A related challenge was scoping: with 24 hours, the discipline was deciding what to fake (the underlying data science) versus what to make genuinely real (the experience and the economic logic behind every number).

5. What I would add with more time

- **Real data ingestion:** let a retailer upload point-of-sale history and train the elasticity/baseline estimates on their actual sales.
- **Side-by-side comparison:** evaluate several products or several discount levels at once.
- **Seasonality and external factors:** richer timing effects (holidays, weather, paydays).
- **Automatic discount optimizer:** surface the single best discount and expected profit up front (the profit chart already hints at this).
- **Accounts and history:** saved promotions, outcomes tracking, and post-promo actuals vs. the prediction to close the learning loop.

6. Assumptions I made

- Simulated intelligence is acceptable per the brief; the goal is a sound, believable experience rather than a trained model.
- Each product's baseline weekly volume, price, cost, and elasticity are representative placeholder values standing in for real historical data.
- Lift responds roughly linearly to discount within the modeled range; real demand curves are more complex and would be learned from data in production.
- Cannibalization is modeled as a rising share of incremental volume; in production this would be estimated per product and category.
- A single promoted product is evaluated at a time (no multi-item baskets in the POC).